

Tratamento de Exceções – Parte 2

A segunda forma de tratar um erro, é delegar ele para quem chamou o nosso método, isto é, passar para a frente.

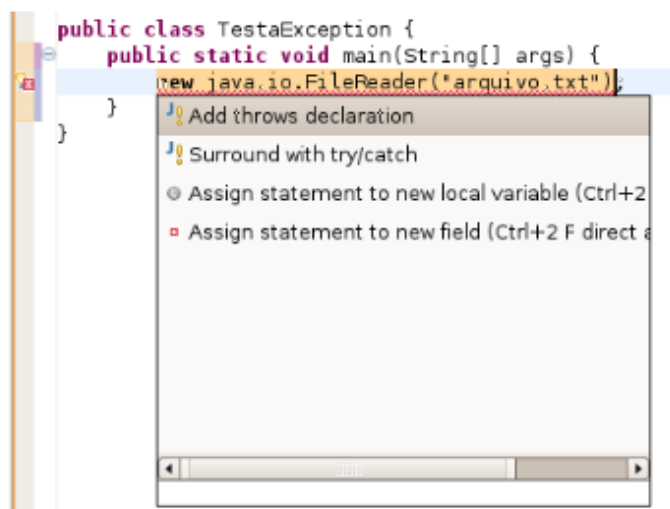
```
public static void metodo() throws java.io.FileNotFoundException {  
    new java.io.FileInputStream("arquivo.txt");  
}
```

No NetBeans é bem simples fazer tanto um try/catch como um throws:

Tente digitar esse código no NetBeans:

```
public class TestaException {  
    public static void main(String[] args) {  
        new java.io.FileInputStream("arquivo.txt");  
    }  
}
```

O NetBeans vai reclamar :



E você tem duas opções:

1. Add throws declaration, que vai gerar:

```
public class TestaException {  
    public static void main(String[] args) throws FileNotFoundException {  
        new java.io.FileInputStream("arquivo.txt");  
    }  
}
```

2. Surround with try/catch, que vai gerar:

```

public class TestaException2 {
    public static void main(String[] args) {
        try {
            new java.io.FileInputStream("arquivo.txt");
        } catch (FileNotFoundException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
    }
}

```

No início, existe uma grande tentação de sempre passar o problema pra frente para outros o tratarem. Pode ser que faça sentido, dependendo do caso, mas não até o main, por exemplo. Acontece que quem tenta abrir um arquivo sabe como lidar com um problema na leitura. Quem chamou um método no começo do programa pode não saber ou, pior ainda, tentar abrir cinco arquivos diferentes e não saber qual deles teve um problema!

Não há uma regra para decidir em que momento do seu programa você vai tratar determinada exceção. Isso vai depender de em que ponto você tem condições de tomar uma decisão em relação àquele erro. Enquanto não for o momento, você provavelmente vai preferir delegar a responsabilidade para o método que te invocou.

EXERCÍCIO

Imagine uma classe que tem um método principal **main** que tem como seu único objetivo alterar todas as letras de uma frase para maiúsculas utilizando o método **toUpperCase()** da classe String, caso a frase esteja nula e se tente usar o método **toUpperCase()** na mesma será lançada uma exceção de **NullPointerException**.

Primeiro vamos ver como ficaria a tal classe sem a utilização do **try/catch**.

Exemplo de código sem try-catch

```

public class aumentaFrase {
    public static void main(String args[])
    {
        String frase = null;
        String novaFrase = null;
        novaFrase = frase.toUpperCase();
        System.out.println("Frase antiga: "+frase);
        System.out.println("Frase nova: "+novaFrase);
    }
}

```

Quando este código for executado, o mesmo lançará uma **NullPointerException**, como poder ser visto na saída do console quando executamos tal programa.

Saída gerada pelo programa sem try-catch

```

Exception in thread "main" java.lang.NullPointerException
at aumentaFrase.main(aumentaFrase.java:15)

```

Ou seja, o mesmo tentou acessar um atributo de um objeto que estava nulo. Para ajudar a melhorar a situação, deve-se usar o try/catch.

Uma solução do o TRY/CATCH

```
public static void main(String args[])
{
    String frase = null;
    String novaFrase = null;
    try
    {
        novaFrase = frase.toUpperCase();
    }
    catch(NullPointerException e) //CAPTURA DA POSSÍVEL exceção.
    {
        //TRATAMENTO DA exceção
        System.out.println("O frase inicial está nula, para solucionar
tal o problema, foi lhe atribuido um valor default.");
        frase = "Frase vazia";
        novaFrase = frase.toUpperCase();
    }
    System.out.println("Frase antiga: "+frase);
    System.out.println("Frase nova: "+novaFrase);
}
```

Quando este código for executado, o mesmo lançará uma `NullPointerException`, porém esta exceção será tratada desta vez, sendo a mesma capturada pelo **catch{}** e dentro deste bloco as devidas providências são tomadas. Neste caso é atribuído um valor default à variável **frase**.

Comando finally

Imagine a seguinte situação: foi aberta uma conexão com o banco de dados para realizar determinada ação, e no meio deste processo seja lançada alguma exceção, como por exemplo, `NullPointerException` ao tentar manipular um determinado atributo de um objeto. Neste caso seria necessário que mesmo sendo lançada uma exceção no meio do processo a conexão fosse fechada. Um outro exemplo bom seria a abertura de determinado arquivo para escrita no mesmo, e no meio deste processo é lançada uma exceção por algum motivo, o arquivo não seria fechado, o que resultaria em deixar o arquivo aberto.

Quando uma exceção é lançada e é necessário que determinada ação seja tomada mesmo após a sua captura, utilizamos a palavra reservada **finally**.

Sintaxe:

Listagem 6: Sintaxe de uso do bloco finally

```
try
{
    //trecho de código que pode vir a lançar uma exceção
}
catch(tipo_excecao_1 e)
{
    //ação a ser tomada
```

```

}
catch(tipo_excecao_2 e)
{
    //ação a ser tomada
}
catch(tipo_excecao _n e)
{
    //ação a ser tomada
}
finally
{
    //ação a ser tomada
}

```

Exemplo:

Listagem 7: Programa aumentaFrase com bloco finally

```

public class aumentaFrase {
    public static void main(String args[])
    {
        String frase = null;
        String novaFrase = null;
        try
        {
            novaFrase = frase.toUpperCase();
        }
        catch(NullPointerException e)
        {
            System.out.println("O frase inicial está nula, para
solucional tal o problema, foi lhe atribuido um valor default.");
            frase = "Frase vazia";
        }
        finally
        {
            novaFrase = frase.toUpperCase();
        }
        System.out.println("Frase antiga: "+frase);
        System.out.println("Frase nova: "+novaFrase);
    }
}

```

Quando este código fosse executado, o mesmo lançaria uma `NullPointerException`, porém esta exceção será tratada desta vez, sendo a mesma capturada pelo `catch{}` e dentro deste bloco as devidas providências são tomadas. Neste caso é atribuído um valor default à variável `frase`. Neste exemplo, mesmo o código lançando uma exceção durante a sua execução e a mesma sendo capturada pelo `catch`, uma determinada ação será tomada no bloco `finally`, neste caso tanto com a exceção ou não será executada a linha “`novaFrase = frase.toUpperCase();`”, tornando todas letras da frase maiúsculas. A saída deste programa seria a seguinte:

Listagem 8: Saída do programa com bloco finally

array4

Comandos throw e throws

Imagine uma situação em que não é desejado que uma exceção seja tratada na própria classe ou método, mas sim em outro que venha lhe chamar. Para solucionar tal situação utilizamos o comando throws na assinatura do método com a possível exceção que o mesmo poderá a vir lançar.

Sintaxe:

Listagem 9: Sintaxe de declaração de método com definição de exceções

```
tipo_retorno nome_metodo() throws tipo_exceção_1, tipo_exceção_2, tipo_exceção_n
{
...
}
```

Onde:

- tipo_retorno – Tipo de retorno do método.
- nome_metodo() - Nome do método que será utilizado.
- tipo_exceção_1 a tipo_exceção_n – Tipo de exceções separadas por virgula que o seu método pode vir a lançar.

Exemplo:

Listagem 10: Definição de exceções que um método pode gerar

```
public class TesteString {
    private static void aumentarLetras() throws NullPointerException
//lançando exceção
    {
        String frase = null;
        String novaFrase = null;
        novaFrase = frase.toUpperCase();
        System.out.println("Frase antiga: "+frase);
        System.out.println("Frase nova: "+novaFrase);
    }

    public static void main(String args[])
    {
        try
        {
            aumentarLetras();
        }
        catch(NullPointerException e)
        {
            System.out.println("Ocorreu um NullPointerException ao
executar o método aumentarLetras() "+e);
        }
    }
}
```

Neste exemplo será lançada uma exceção no método aumentarLetras():

Listagem 11: Definição da exceção gerada pelo método aumentarLetras

```
private static void aumentarLetras() throws NullPointerException
```

E o mesmo será tratado no método main().

Listagem 12

: Aplicação da exceção definida

```
...
try
{
    aumentarLetras();
}
catch(NullPointerException e)
{
    System.out.println("Ocorreu um NullPointerException ao executar o método
aumentarLetras() "+e);
}
...
```

Saída:

Listagem 13: Saída do programa atualizado

Ocorreu um NullPointerException ao executar o método aumentarLetras()
java.lang.NullPointerException.

Agora imagine o caso em que seja necessário lançar uma exceção padrão ao invés de uma específica. Para resolver este problema, utilizamos o comando throw dentro do bloco catch que desejamos converter a exceção.

Sintaxe:

Listagem 14: Sintaxe de uso do comando throw

```
try
{
    //...
}
catch(tipoExcessão_1 e)
{
    throw new novoTipoExcecao(e);
}
```

Onde:

- tipoExcessão_1 e – Tipo de exceção que pode ser capturada pelo bloco catch.
- NovoTipoExcecao – Tipo de exceção que será lançada.

Exemplo:

Listagem 15: Exemplo de uso do comando throw

```
public class TesteString {
    private static void aumentarLetras() throws Exception //lançando exceção
    {
        String frase = null;
        String novaFrase = null;
        try
        {
            novaFrase = frase.toUpperCase();
        }
        catch(NullPointerException e)
```

```

        {
            throw new Exception(e);
        }
        System.out.println("Frase antiga: "+frase);
        System.out.println("Frase nova: "+novaFrase);
    }
    public static void main(String args[])
    {
        try
        {
            aumentarLetras();
        }
        catch(Exception e)
        {
            System.out.println("Ocorreu uma exceção ao executar o
método aumentarLetras() "+e);
        }
    }
}

```

Neste exemplo será lançada uma `NullPointerException` e a mesma será convertida para `Exception` e relançada como `Exception` no método `aumentarLetras()` e, por fim, a mesma é tratada no método `main()`.

Saida:

Listgem 16: Saída do programa atualizada

```
Ocorreu uma exceção ao executar o método aumentarLetras() java.lang.Exception:
java.lang.NullPointerException
```

Criando exceções

Assim como qualquer objeto, em Java também é possível criar suas próprias exceções. Imagine um cenário em que nenhuma exceção existente faça sentido para ser lançada por você.

Por exemplo, imagine que por algum motivo você precisa que uma exceção seja lançada quando a letra “B” ou “b” não existe e determinada frase, como não existe nenhuma exceção específica para este caso será necessário criar uma exceção.

Criando uma exceção para ser lançada toda vez que uma letra “B” ou “b” não é encontrada em uma determinada frase.

Listagem 17: Exemplo de exceção customizada

```

public class SemLetraBException extends Exception {
    @Override
    public String getMessage(){
        return "Não existe letra B em sua frase";
    }
}

```

Toda exceção criada deve estender `Exception`, neste exemplo foi sobrescrito o método `getMessage()`, que é exibida no prompt toda vez que a exceção é lançada.

Utilizando a exceção

Abaixo segue um exemplo que é utilizada a exceção criada acima.

Listagem 16: Utilizando a exceção customizada

```
public class TesteExcecao {  
    public static void main(String args[]) throws SemLetraBException  
    {  
        String frase = "Sou um teste!";  
        if(!frase.contains("b") || !frase.contains("B"))  
            throw new SemLetraBException();  
    }  
}
```

Quando o programa acima fosse executado, uma exceção do tipo `SemLetraBException()` seria lançada. Abaixo está a saída exibida no prompt:

Listagem 17: Saída do método com a mensagem da exceção customizada

```
Exception in thread "main" SemLetraBException: Não existe letra B ou b em sua frase at  
TesteExcecao.main(TesteExcecao.java:8)
```